

FERDOWSI UNIVERSITY OF MASHHAD

Department of Computer Engineering

FORMAL VERIFICATION OF A LOOP INVARIANT

A Step-by-Step Lean 4 Proof for Phase 3, Option 2

Authors

Ehsan Soveizi & Mahdi Rashidi

Course

Theory of Languages and Machines - Project 2: Model Checker

Chosen theorem

Loop Invariant (Option 2)

July 2026

Abstract

This report presents the design and formal verification of a generic loop-invariant theorem for a selected operational-semantic subset of the Hash language. The formalization is written in Lean 4 and uses relational big-step semantics. The central theorem is parametric in an arbitrary state predicate P : if P holds before a while loop, and every terminating execution of the loop body preserves P , then P holds after every terminating execution of the complete loop. The proof proceeds by structural induction on the BigStep derivation of the loop execution. Because BigStep is an indexed inductive family, the proof first establishes a strengthened helper statement in which the command index is generalized and connected to a while command by an equality hypothesis. This design allows Lean to perform induction cleanly and makes the proof cases correspond directly to the inference rules of the semantics.

The report explains the choice of the statement subset, the semantic definitions, the exact logical meaning of the theorem, the reason for induction on execution derivations rather than program syntax, the role of constructor disjointness in impossible cases, and the two substantive loop cases. It also distinguishes partial correctness from termination, documents assumptions and limitations, and provides a worked invariant example and a presentation-oriented question-and-answer guide.

Main contribution

A reusable theorem showing preservation of any predicate P across every terminating execution of a while loop, with a proof structure that mirrors the BigStep inference rules and remains independent of any particular program or variable.

Contents

1. Introduction and project context.....	3
2. Why this subset of statements was selected.....	3
3. Formal semantic model.....	4
4. The loop-invariant theorem.....	6
5. Proof strategy and the strengthened helper theorem.....	7
6. Detailed proof walkthrough.....	9
7. Correctness, completeness, and proof obligations.....	11
8. Worked example.....	12
9. Design decisions, assumptions, and limitations.....	12
10. Oral-presentation guide.....	13
11. Conclusion.....	14
Appendix A. Complete Lean 4 source.....	15
Appendix B. Submission checklist.....	17
References.....	18

1. Introduction and Project Context

Phase 3 of the project moves from model checking a particular finite Promela model to proving a language-level property from operational semantics. The project brief asks students to define the operational semantics of a selected Hash subset and formally prove one of three theorems in Lean or Coq. Option 2 is the loop-invariant theorem. Its intended meaning is that a predicate on states remains true after a complete loop execution when it is initially true and the body preserves it.

This theorem is a natural choice because it connects three major topics in formal methods: operational semantics, induction on derivations, and program correctness. It is also more general than proving a property of one specific loop. The predicate P is universally quantified, so the theorem applies to non-negativity, bounds, conservation properties, relationships among variables, array-index safety conditions represented in the state, or any other proposition expressible as $\text{State} \rightarrow \text{Prop}$.



Figure 1. From the selected Hash subset to the generic loop-invariant proof.

The formal statement is a partial-correctness result. It assumes that a BigStep derivation of the loop exists; therefore, the loop has terminated in the final state described by that derivation. The theorem then proves that P holds in that final state. It does not claim that the loop always terminates. This distinction is essential and should be stated explicitly in both the report and the oral presentation.

Terminology

The proof establishes partial correctness: if the loop terminates, the invariant holds afterward. Total correctness would additionally require a separate termination argument, normally based on a well-founded decreasing measure.

2. Why This Subset of Statements Was Selected

The Phase 3 formalization uses four statement constructors: assignment, sequential composition, conditional branching, and while loops. This is a deliberate semantic core rather than an arbitrary reduction of the Hash language. The subset is large enough to express meaningful imperative programs and nested control flow, while remaining small enough that every inference rule and every proof obligation can be explained rigorously.

2.1 Selection criteria

- **Direct relevance to the theorem.** A loop-invariant theorem fundamentally requires while. The loop body must still be able to modify the state and contain realistic control flow, which requires assign, seq, and ifElse.
- **Alignment with standard natural semantics.** These four constructs have canonical big-step rules. Their constructors map directly to the standard Assign, Seq, If-True, If-False, While-False, and While-True inference rules.
- **Sufficient expressive power.** The body of a while loop can be a sequence, contain conditionals, and even contain nested loops. Therefore the theorem is not restricted to a trivial single-assignment body.
- **Proof transparency.** Every BigStep constructor corresponds to one induction case. This one-to-one correspondence makes the proof auditable and easier to defend during the presentation.
- **Modular extensibility.** Additional Hash constructs can later be assigned semantics without changing the central idea of the theorem. The core proof remains a stable foundation.

- **Controlled scope.** The project asks for one complete formal proof. A smaller, semantically coherent core is preferable to a broad but partially specified language fragment.

2.2 Why other constructs were not included

Excluded feature	Reason for exclusion from the Phase 3 core
for / baraye	It is syntactic sugar for initialization followed by while with an update. A separate rule is unnecessary for the invariant theorem.
break / shekan	It introduces abrupt loop exit. The semantics would need an additional execution-result type or a control-flow signal, and the theorem would require extra exit cases.
continue / edame	It skips the remaining body and returns to the guard. This changes what counts as one body execution and requires a richer control-effect semantics.
try/catch	Exceptions create normal and exceptional outcomes. A simple State-to-State BigStep relation is no longer sufficient without encoding control outcomes.
classes and methods	They require environments, object stores, call frames, and method invocation rules that are orthogonal to the selected theorem.
variable declarations and types	The proof concerns state transformation after execution. Static typing and declaration rules are important, but not required for this semantic preservation result.

Excluding these constructs does not invalidate the theorem. It defines the exact language fragment over which the theorem is proved. Formal verification always requires a clearly stated semantic scope. A strong report should identify that scope explicitly rather than suggesting that the proof covers constructs for which no operational rules were defined.

Important defense point

The subset was not selected to make the loop case trivial. The body remains an arbitrary Stmt and may contain sequences, branches, assignments, and nested while loops. The theorem is therefore structurally general within the declared semantic fragment.

3. Formal Semantic Model

The proof depends on a precise semantic model. Each definition has a distinct role: State represents memory, Expr represents arithmetic and comparison expressions, EvalExpr specifies expression evaluation, Stmt represents commands, update models assignment, and BigStep defines complete terminating statement executions.

3.1 Variables and states

```
abbrev Var := String
abbrev State := Var -> Int
```

Listing 1. Variables and program states.

A state is a total function from variable names to integers. For any state s and variable x , the expression $s\ x$ is the current integer value of x . A functional representation avoids an explicit finite map implementation and makes the semantic rules concise. Variables not explicitly updated retain their previous values.

3.2 Expressions and relational evaluation

The expression language contains numeric literals, variables, addition, multiplication, less-than comparison, and equality comparison. Comparison results are encoded as integers: 1 represents true and 0 represents false. Statement guards interpret any nonzero value as true, which is consistent with the rules used for conditionals and while loops.

```
inductive EvalExpr : Expr -> State -> Int -> Prop where
| num ...
| var ...
| add ...
| mul ...
| lt ...
| eq ...
```

Listing 2. Relational evaluation of expressions.

EvalExpr is a relation rather than a function. The proposition EvalExpr $e\ s\ v$ means that expression e evaluates to value v in state s . The loop-invariant theorem does not depend on expression determinism: it only uses the evaluation evidence already present in a BigStep derivation. Nevertheless, the supplied expression rules are deterministic because each expression constructor uniquely determines its result from the results of its subexpressions.

3.3 Statements and state update

```
inductive Stmt where
| assign : Var -> Expr -> Stmt
| seq    : Stmt -> Stmt -> Stmt
| ifElse : Expr -> Stmt -> Stmt -> Stmt
| while  : Expr -> Stmt -> Stmt

def update (s : State) (x : Var) (v : Int) : State :=
  fun y => if y == x then v else s y
```

Listing 3. Statement syntax and functional state update.

The function update $s\ x\ v$ constructs a new state. When queried at x it returns v ; for every different variable y it returns the old value $s\ y$. The original state is not mutated. This functional interpretation is standard in proof assistants and makes the assignment rule mathematically explicit.

3.4 Big-step semantics

BigStep $S\ sIn\ sOut$ means that statement S , starting in state sIn , terminates in state $sOut$. Each constructor is an inference rule. Because BigStep describes only terminating executions, a non-terminating loop has no finite BigStep derivation.

BigStep constructor	Semantic meaning
assign	Evaluate the expression and update one variable.
seq	Execute the first command to an intermediate state, then execute the second command.
ifTrue	Evaluate the guard to a nonzero value and execute the then branch.

BigStep constructor	Semantic meaning
ifFalse	Evaluate the guard to zero and execute the else branch.
whileFalse	If the guard evaluates to zero, stop immediately and preserve the state.
whileTrue	If the guard is nonzero, execute the body once and then execute the same loop recursively.

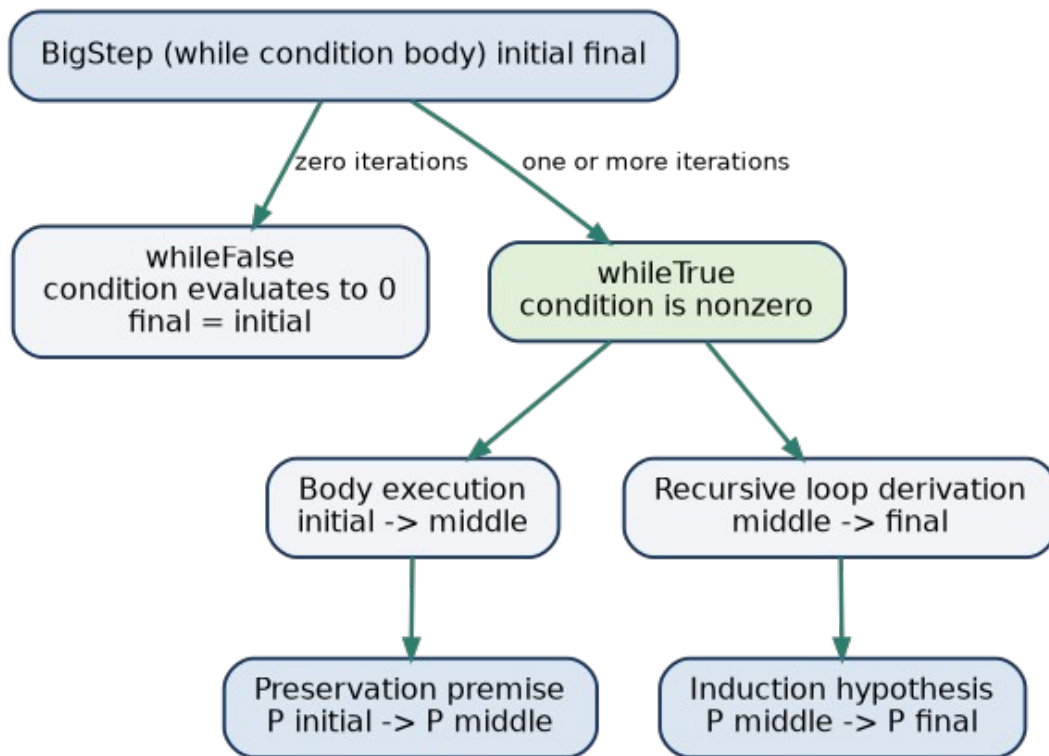


Figure 2. The two possible final rules for a terminating while-loop derivation.

4. The Loop-Invariant Theorem

```

theorem loop_invariant
  {P : State -> Prop}
  {condition : Expr}
  {body : Stmt}
  {initial final : State}
  (hInitial : P initial)
  (hPreserves :
    forall {before after : State},
      P before ->
        BigStep body before after ->
          P after)
  (hLoop : BigStep (Stmt.while condition body) initial final) :
  P final := by
  ...

```

Listing 4. Formal theorem statement.

4.1 Meaning of the quantified objects

Item	Interpretation
$P : \text{State} \rightarrow \text{Prop}$	An arbitrary predicate on states. The theorem is not tied to one concrete invariant.
<code>condition : Expr</code>	The loop guard.
<code>body : Stmt</code>	An arbitrary statement in the selected semantic subset.
<code>initial, final : State</code>	The input and output states of one terminating loop execution.
<code>hInitial</code>	Evidence that P is true before the loop.
<code>hPreserves</code>	Evidence that every terminating execution of the body maps a P -state to another P -state.
<code>hLoop</code>	A complete terminating BigStep derivation of the while loop.
<code>P final</code>	The required postcondition: the invariant remains true after the loop.

4.2 Logical form

In mathematical notation, the theorem states:

Loop invariant rule

$P(\text{initial}) \text{ AND } [\text{for all before, after: } P(\text{before}) \text{ AND } \langle \text{body, before} \rangle \Rightarrow \text{after} \text{ implies } P(\text{after})] \text{ AND } \langle \text{while condition body, initial} \rangle \Rightarrow \text{final} \text{ IMPLIES } P(\text{final}).$

The preservation premise is universally quantified over the starting and ending states of the body. This is crucial. A premise that only showed preservation for one particular state would not support an arbitrary number of iterations. The theorem needs to apply the same preservation argument after each completed body execution.

4.3 Partial correctness and termination

The theorem assumes `hLoop`. Therefore, it reasons about a loop execution that already has a finite terminating derivation. A diverging loop is outside the premise because no BigStep proof of its final state exists. The theorem is therefore sound even for loops that may fail to terminate on some states: it makes a claim only about terminating executions.

The preservation premise also speaks only about terminating body executions. If the body itself diverges, there is no BigStep body before after proof, so the implication is not applicable. This is exactly the standard partial-correctness interpretation of big-step semantics.

5. Proof Strategy and the Strengthened Helper Theorem

5.1 Why induction is performed on the derivation

The theorem concerns an execution described by `hLoop`, not merely the syntactic shape of the loop. The correct induction object is therefore the BigStep derivation. Induction on the syntax `Stmt.while condition`

body would not reveal how many iterations occurred or which final semantic rule was used. By contrast, induction on the derivation exposes whether execution ended by `whileFalse` or proceeded by `whileTrue`, and in the latter case provides a structurally smaller derivation for the remaining loop iterations.

5.2 Why a direct induction may fail in Lean

`BigStep` is an indexed inductive family: its first index is the command being executed. In `hLoop`, that index is already fixed to `Stmt.while` condition body rather than being a free variable. Lean may reject a direct induction because the index in the target is not a variable. This is a dependent-induction issue, not a weakness in the mathematical theorem.

Typical Lean issue

A direct induction on `hLoop` can produce an error similar to: "Invalid target: Index in target's type is not a variable." The solution is to generalize the command index before induction.

5.3 The strengthened helper statement

```
have invariant_for_any_derivation :
  forall {command : Stmt} {start stop : State},
    BigStep command start stop ->
    forall {b : Expr} {loopBody : Stmt},
      command = Stmt.while b loopBody ->
      P start ->
      (forall {before after : State},
        P before ->
        BigStep loopBody before after ->
        P after) ->
      P stop := by
...

```

Listing 5. Generalized helper theorem used as the induction motive.

The helper theorem generalizes `command`, `start`, and `stop`. It then introduces an equality requiring `command` to be a `while` statement. This preserves the intended theorem while giving induction a variable `command` index. During each constructor case, the equality determines whether that constructor could actually have produced a `while` command.

This strengthening is a common proof-engineering technique: formulate an induction hypothesis general enough to survive dependent pattern matching. The outer theorem is then recovered by instantiating the helper with `hLoop` and reflexivity.

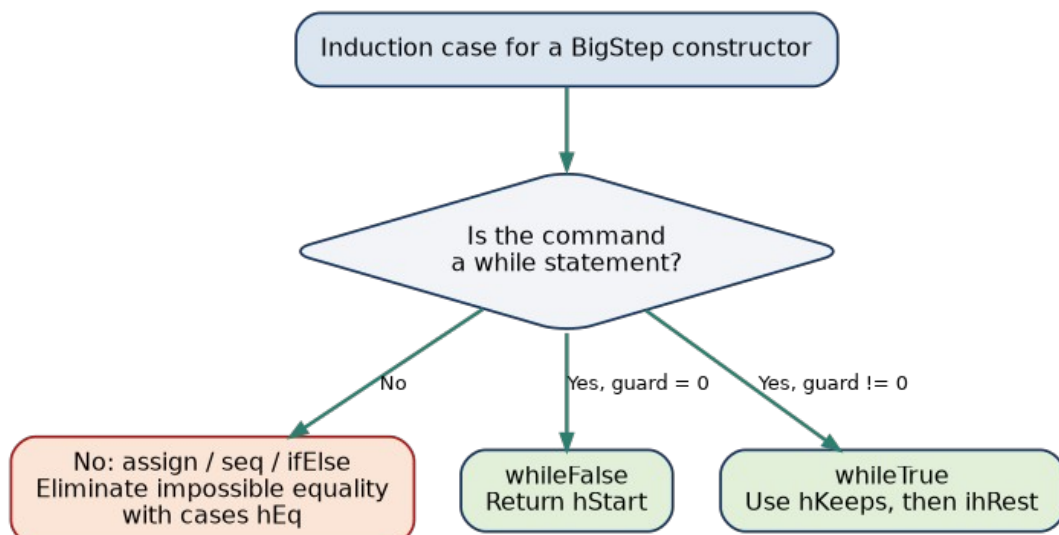


Figure 3. Classification of induction cases in the strengthened proof.

5.4 Structural decrease

In the `whileTrue` constructor, the derivation contains a premise `hRest` for the execution of the remaining loop from the intermediate state. `hRest` is an immediate sub-derivation of the complete `whileTrue` derivation. Therefore, the induction hypothesis `ihRest` is structurally justified. The proof does not recurse on program states or attempt to guess the number of iterations; it follows the finite proof tree supplied by `BigStep`.

6. Detailed Proof Walkthrough

After introducing `command`, `start`, `stop`, and the execution derivation `h`, the proof performs induction on `h`. All six `BigStep` constructors must be considered. Four cases are impossible because their `command` constructors cannot equal `Stmt.while`. The remaining two cases correspond to zero iterations and one-or-more iterations.

6.1 Assignment case

```
| assign hEval =>
  intro b loopBody hEq
  cases hEq
```

Listing 6. Assignment case.

In this branch, `command` has the form `Stmt.assign x e`. The hypothesis `hEq` claims that this assignment is equal to `Stmt.while b loopBody`. Since `assign` and `while` are distinct constructors of the inductive type `Stmt`, the equality has no constructor. The tactic `cases hEq` eliminates the impossible branch by constructor disjointness.

6.2 Sequence case

```
| seq hFirst hSecond ihFirst ihSecond =>
  intro b loopBody hEq
  cases hEq
```

Listing 7. Sequential-composition case.

The same argument applies. A `Stmt.seq` command cannot be equal to a `Stmt.while` command. The induction hypotheses for the two sequence premises are not needed because the branch is impossible before any semantic reasoning about the premises is required.

6.3 Conditional cases

```
| ifTrue hEval hNonzero hThen ihThen =>
  intro b loopBody hEq
  cases hEq

| ifFalse hEval hElse ihElse =>
  intro b loopBody hEq
  cases hEq
```

Listing 8. Conditional cases.

Both conditional constructors have `Stmt.ifElse` as their `command` index. Their equality with a `while` command is impossible, so both branches close by `cases hEq`. This is a syntactic impossibility established by the inductive constructors of `Stmt`; it does not rely on evaluating the condition.

6.4 Base case: whileFalse

```
| whileFalse hCondition =>
  intro b loopBody hEq hStart hKeeps
  cases hEq
  exact hStart
```

Listing 9. Zero-iteration loop case.

The whileFalse rule states that if the condition evaluates to zero, the loop terminates without executing the body and without changing the state. Consequently, stop is definitionally the same state as start after the equality is resolved. The proof already has $hStart : P \text{ start}$, so $\text{exact } hStart$ proves $P \text{ stop}$. The preservation premise $hKeeps$ is correctly unused because the body was not executed.

6.5 Inductive case: whileTrue

```
| whileTrue hCondition hNonzero hBody hRest ihBody ihRest =>
  intro b loopBody hEq hStart hKeeps
  cases hEq

  have hMiddle : P _ := hKeeps hStart hBody
  exact ihRest rfl hMiddle hKeeps
```

Listing 10. One-or-more-iteration loop case.

This is the only nontrivial branch. The whileTrue rule supplies three semantic facts: the guard is nonzero, the body executes from the starting state to an intermediate state, and the same loop executes from that intermediate state to the final state.

1. **Resolve the command equality.** $\text{cases } hEq$ rewrites the generalized command and loop body so that the local preservation premise applies to the actual body in $hBody$.
2. **Preserve P through one body execution.** $hKeeps \ hStart \ hBody$ proves P for the intermediate state. The placeholder in $P _$ is inferred from the output state of $hBody$.
3. **Apply the induction hypothesis to the remaining loop.** $ihRest$ concerns the structurally smaller derivation $hRest$. It receives rfl to show that its command is the required while loop, $hMiddle$ as the invariant at the recursive starting state, and $hKeeps$ as the same body-preservation property.
4. **Conclude P final.** The result of $ihRest$ is exactly the goal for the final state.

The induction hypothesis $ihBody$ is unused because the theorem does not need to inspect the internal derivation of the body. Preservation of the body is already assumed in $hKeeps$. In contrast, $ihRest$ is essential because it represents the remaining iterations of the same loop.

6.6 Returning to the original theorem

```
exact invariant_for_any_derivation hLoop rfl hInitial hPreserves
```

Listing 11. Instantiation of the helper theorem.

The outer theorem applies the helper to the original loop execution. The equality argument is rfl because the command in $hLoop$ is already syntactically $\text{Stmt.while condition body}$. The remaining arguments are exactly the initial invariant and the preservation premise. This final line discharges $P \text{ final}$.

6.7 Case coverage summary

Constructor	Reasoning	Key Lean step
assign	Impossible equality: $\text{assign} \neq \text{while}$	$\text{cases } hEq$
seq	Impossible equality: $\text{seq} \neq \text{while}$	$\text{cases } hEq$

Constructor	Reasoning	Key Lean step
ifTrue	Impossible equality: ifElse != while	cases hEq
ifFalse	Impossible equality: ifElse != while	cases hEq
whileFalse	Zero iterations; state unchanged	exact hStart
whileTrue	Preserve P once, then use IH for remaining iterations	hKeeps + ihRest

7. Correctness, Completeness, and Proof Obligations

7.1 Why the proof is sound

Every conclusion is obtained from a constructor premise, an explicitly supplied hypothesis, or an induction hypothesis associated with a strict sub-derivation. No semantic fact is assumed without evidence. Impossible cases are eliminated only when Lean has an equality between different constructors. The whileFalse result follows from the semantic rule that preserves the state, and the whileTrue result follows from one use of preservation plus the induction hypothesis for the recursive loop derivation.

7.2 Why the proof is complete

BigStep has exactly six constructors, and the induction contains one branch for each constructor. For a derivation whose command is a while statement, the only semantically possible final constructors are whileFalse and whileTrue. Both are handled. Therefore, every finite terminating execution of a while loop represented by the defined semantics is covered.

7.3 The role of the guard

The current preservation premise is stronger than the weakest possible premise because it requires the body to preserve P whenever the body executes from a P-state, even if no guard fact is supplied. A guard-sensitive formulation could additionally require evidence that condition is nonzero. The stronger form is conventional, simpler, and matches the project wording that the body preserves the invariant. It also explains why hCondition and hNonzero are not used in the whileTrue branch.

```
/* Optional weaker, guard-sensitive premise */
forall {before after : State} {v : Int},
  P before ->
  EvalExpr condition before v ->
  v != 0 ->
  BigStep body before after ->
  P after
```

Listing 12. An alternative preservation premise (illustrative only).

The submitted theorem should retain one formulation consistently. The existing stronger premise is correct and easier to present. The alternative is useful only to demonstrate awareness of logical strength.

7.4 Nested loops and arbitrary bodies

The theorem does not require body to be an assignment or sequence. It quantifies over an arbitrary Stmt. If body contains a nested loop, hPreserves must establish that the complete terminating body execution preserves P. Once that obligation is available, the outer loop proof is unchanged. This is a modularity advantage: the invariant theorem treats the body through its semantic behavior rather than its syntax.

7.5 Trust boundary

The theorem is correct relative to the declared definitions of `Expr`, `EvalExpr`, `Stmt`, `update`, and `BigStep`. This is the normal meaning of a formal semantic proof. If additional language features are introduced, their semantics must be added and the scope of the theorem reassessed. The proof does not automatically apply to an implementation whose behavior differs from these rules.

8. Worked Example

Consider a simple loop that increments `x` while `x` is less than 10. Let the invariant be non-negativity of `x`.

```
condition = Expr.lt (Expr.var "x") (Expr.num 10)
body      = Stmt.assign "x"
           (Expr.add (Expr.var "x") (Expr.num 1))

P s := s "x" >= 0
```

Listing 13. Example loop and invariant.

Suppose the initial state satisfies `initial "x" >= 0`. To use the generic theorem, the only program-specific proof obligation is `hPreserves`: whenever `x` is nonnegative before the assignment and the assignment terminates, `x` is nonnegative afterward. The assignment evaluates `x + 1` and updates `x` to that value, which is still nonnegative.

5. **Initial state.** From `hInitial`, `P` initial holds.
6. **First successful guard evaluation.** If `x < 10` evaluates to a nonzero value, `whileTrue` executes the body.
7. **Body preservation.** After `x := x + 1`, the intermediate state satisfies `P`.
8. **Remaining iterations.** The induction hypothesis applies from the intermediate state.
9. **Termination.** When the guard evaluates to zero, `whileFalse` returns the current state, which still satisfies `P`.
10. **Final result.** The generic `loop_invariant` theorem concludes `P` final without needing to know the exact number of iterations.

What the generic theorem buys us

Once `hPreserves` has been proved for this body, the loop proof does not repeat arithmetic reasoning for each iteration. Structural induction propagates the invariant through an arbitrary finite number of iterations.

The example also illustrates why the theorem is parametric. Replacing `P` with a different predicate changes the preservation obligation but not the loop proof. For example, `P` could express `x <= 10`, a relation such as `x + y = c`, or a conjunction of several safety conditions.

9. Design Decisions, Assumptions, and Limitations

Design point	Current choice	Consequence
Boolean representation	Comparisons return 1 or 0; statement guards accept any nonzero value as true.	Consistent with the semantics, but should be documented.
Relational <code>EvalExpr</code>	Expression evaluation is a proposition rather than a recursive function.	Integrates naturally with relational <code>BigStep</code> ; determinism can be proved separately.

Design point	Current choice	Consequence
Big-step semantics	Only terminating complete executions are represented.	Ideal for partial correctness; divergence is represented by absence of a derivation.
Strong preservation premise	The body preserves P without explicitly requiring the guard to be true.	Correct and conventional, though stronger than necessary.
Functional state	State is $\text{Var} \rightarrow \text{Int}$.	Simple and extensional; all variables always have a value.
Core statement subset	No abrupt control effects or exceptions.	Keeps State-to-State BigStep adequate and the theorem precise.
No termination proof	hLoop is assumed.	The theorem proves partial, not total, correctness.
No link to generated Promela	Phase 3 semantics is an independent formal model.	A compiler-correctness theorem would be a separate, larger task.

Explicitly documenting limitations improves the quality of a formal report. It shows that the scope was chosen intentionally and that the theorem is not being overclaimed. The correct claim is strong within its scope: it applies to every predicate P , every loop condition, every body in the subset, and every terminating BigStep execution.

10. Oral-Presentation Guide

The following questions are likely to arise in a technical evaluation. The answers should be understood rather than memorized.

Q1. Why did you choose these four statements?

They form the standard imperative core needed for a meaningful while-loop theorem. Assignment changes state, sequence composes effects, conditionals provide branching, and while provides iteration. More advanced control effects require richer semantics and are outside the selected Phase 3 scope.

Q2. Why is the theorem generic in P ?

A loop-invariant theorem should be reusable. By quantifying over $P : \text{State} \rightarrow \text{Prop}$, the proof applies to any state property for which initial truth and body preservation can be established.

Q3. Why induction on BigStep rather than on Stmt?

The number of loop iterations is encoded in the execution derivation, not in the syntax of the while statement. The whileTrue premise contains a smaller derivation for the remaining iterations.

Q4. Why is a helper theorem necessary?

BigStep is indexed by the command. The original hLoop fixes that index to a while expression, which can obstruct dependent induction. Generalizing command and adding an equality produces a valid induction motive.

Q5. What does cases hEq do?

It uses constructor disjointness. An assign, seq, or ifElse command cannot equal a while command, so those branches are logically impossible.

Q6. What happens in whileFalse?

The guard is zero, the body is not executed, and the output state equals the input state. Therefore the initial invariant is already the final invariant.

Q7. What happens in whileTrue?

The body executes once. hPreserves transfers P to the intermediate state, and ihRest transfers it through the remaining loop execution to the final state.

Q8. Why is ihBody unused?

The body-preservation hypothesis already summarizes exactly the property needed from the complete body execution. The proof only needs induction over the recursively repeated loop.

Q9. Does this prove termination?

No. hLoop assumes a terminating derivation. The theorem proves partial correctness: every terminating execution ends in a state satisfying P .

Q10. Could the body contain another while loop?

Yes. body is an arbitrary Stmt. The outer theorem only requires a proof that the complete body execution preserves P .

Q11. Is the preservation premise stronger than necessary?

Yes. It does not require the guard to be true. This stronger conventional form is still correct and simplifies the theorem.

Q12. What exactly has been formally verified?

Relative to the defined operational semantics, any terminating execution of a while loop preserves any predicate that is initially true and preserved by each terminating body execution.

11. Conclusion

The formalization provides a correct and complete proof of the loop-invariant theorem for the declared Hash statement subset. The semantic definitions are sufficiently expressive for nontrivial imperative programs, and the proof mirrors the operational rules. The generalized helper theorem resolves the dependent-induction issue caused by the indexed BigStep relation. Constructor disjointness removes the non-loop cases, while the two loop rules establish the base and inductive steps.

The final result is intentionally generic: it is independent of a particular variable, condition, body, number of iterations, or invariant formula. Its exact claim is partial correctness. Whenever a complete terminating loop execution exists, an invariant that is true initially and preserved by every terminating body execution is true in the final state. This is the standard semantic foundation of Hoare-style reasoning about loops.

Final theorem in one sentence

A loop cannot lose a predicate P during a terminating execution when P is true before the loop and each completed execution of the body preserves P .

Appendix A. Complete Lean 4 Source

The following source is the complete formalization supplied for Phase 3. Comments are included to clarify the semantic role of each definition and the structure of the proof.

```

/-
Phase 3 - Formal Proof in Lean 4
Chosen theorem: Loop Invariant (Option 2)

Authors: Ehsan Soveizi & Mahdi Rashidi
-/

namespace LoopInvariantProof

abbrev Var := String
abbrev State := Var -> Int

inductive Expr where
| num : Int -> Expr
| var : Var -> Expr
| add : Expr -> Expr -> Expr
| mul : Expr -> Expr -> Expr
| lt : Expr -> Expr -> Expr
| eq : Expr -> Expr -> Expr
deriving Repr

inductive EvalExpr : Expr -> State -> Int -> Prop where
| num {n : Int} {s : State} :
  EvalExpr (Expr.num n) s n

| var {x : Var} {s : State} :
  EvalExpr (Expr.var x) s (s x)

| add {e1 e2 : Expr} {s : State} {v1 v2 : Int} :
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.add e1 e2) s (v1 + v2)

| mul {e1 e2 : Expr} {s : State} {v1 v2 : Int} :
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.mul e1 e2) s (v1 * v2)

| lt {e1 e2 : Expr} {s : State} {v1 v2 : Int} :
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.lt e1 e2) s
  (if v1 < v2 then 1 else 0)

```

Listing A.1. Complete source, part 1 of 4.

```

| eq {e1 e2 : Expr} {s : State} {v1 v2 : Int} :
  EvalExpr e1 s v1 ->
  EvalExpr e2 s v2 ->
  EvalExpr (Expr.eq e1 e2) s
  (if v1 = v2 then 1 else 0)

inductive Stmt where
| assign : Var -> Expr -> Stmt
| seq : Stmt -> Stmt -> Stmt
| ifElse : Expr -> Stmt -> Stmt -> Stmt
| while : Expr -> Stmt -> Stmt
deriving Repr

def update (s : State) (x : Var) (v : Int) : State :=
  fun y => if y == x then v else s y

```

```

inductive BigStep : Stmt -> State -> State -> Prop where
| assign {x : Var} {e : Expr} {s : State} {v : Int} :
  EvalExpr e s v ->
  BigStep (Stmt.assign x e) s (update s x v)

| seq {S1 S2 : Stmt} {s smid sout : State} :
  BigStep S1 s smid ->
  BigStep S2 smid sout ->
  BigStep (Stmt.seq S1 S2) s sout

| ifTrue {b : Expr} {S1 S2 : Stmt}
  {s sout : State} {v : Int} :
  EvalExpr b s v ->
  v != 0 ->
  BigStep S1 s sout ->
  BigStep (Stmt.ifElse b S1 S2) s sout

| ifFalse {b : Expr} {S1 S2 : Stmt} {s sout : State} :
  EvalExpr b s 0 ->
  BigStep S2 s sout ->
  BigStep (Stmt.ifElse b S1 S2) s sout

| whileFalse {b : Expr} {body : Stmt} {s : State} :
  EvalExpr b s 0 ->
  BigStep (Stmt.while b body) s s

| whileTrue {b : Expr} {body : Stmt}
  {s smid sout : State} {v : Int} :

```

Listing A.2. Complete source, part 2 of 4.

```

  EvalExpr b s v ->
  v != 0 ->
  BigStep body s smid ->
  BigStep (Stmt.while b body) smid sout ->
  BigStep (Stmt.while b body) s sout

theorem loop_invariant
  {P : State -> Prop}
  {condition : Expr}
  {body : Stmt}
  {initial final : State}
  (hInitial : P initial)
  (hPreserves :
    forall {before after : State},
      P before ->
      BigStep body before after ->
      P after)
  (hLoop : BigStep (Stmt.while condition body) initial final) :
  P final := by

have invariant_for_any_derivation :
  forall {command : Stmt} {start stop : State},
    BigStep command start stop ->
    forall {b : Expr} {loopBody : Stmt},
      command = Stmt.while b loopBody ->
      P start ->
      (forall {before after : State},
        P before ->
        BigStep loopBody before after ->
        P after) ->
      P stop := by

intro command start stop h

induction h with

| assign hEval =>
  intro b loopBody hEq
  cases hEq

| seq hFirst hSecond ihFirst ihSecond =>
  intro b loopBody hEq
  cases hEq

```

Listing A.3. Complete source, part 3 of 4.

```

| ifTrue hEval hNonzero hThen ihThen =>
  intro b loopBody hEq
  cases hEq

| ifFalse hEval hElse ihElse =>
  intro b loopBody hEq
  cases hEq

| whileFalse hCondition =>
  intro b loopBody hEq hStart hKeeps
  cases hEq
  exact hStart

| whileTrue hCondition hNonzero hBody hRest ihBody ihRest =>
  intro b loopBody hEq hStart hKeeps
  cases hEq

  have hMiddle : P _ := hKeeps hStart hBody
  exact ihRest rfl hMiddle hKeeps

exact invariant_for_any_derivation hLoop rfl hInitial hPreserves
end LoopInvariantProof

```

Listing A.4. Complete source, part 4 of 4.

Appendix B. Submission Checklist

Check	Evidence expected
Semantic scope is stated	The report explicitly says the theorem covers assign, seq, ifElse, and while.
Expression truth convention is stated	Comparisons return 0 or 1; nonzero guards are treated as true.
BigStep meaning is stated	It describes complete terminating execution from an input state to an output state.
The theorem is translated into words	Each premise and the conclusion are explained.
Partial correctness is distinguished from termination	The report does not claim that every loop terminates.
Induction object is justified	The proof uses structural induction on the BigStep derivation.
Helper theorem is justified	The indexed-family issue and generalized command equality are explained.
All six constructors are covered	Four impossible cases and two loop cases are documented.
whileFalse is explained	Zero iterations preserve the initial state.
whileTrue is explained	hKeeps establishes P middle; ihRest establishes P final.
Unused hypotheses are explained	ihBody, hCondition, and hNonzero are not mistakenly presented as necessary.
A worked invariant example is included	The generic theorem is connected to a concrete program.
Full source is included	The report and submitted .lean file remain consistent.
Compilation is checked in the target Lean environment	The final .lean file is compiled immediately before submission.
Oral defense is prepared	The authors can explain every premise, case, and tactic.

References

- [1] Ferdowsi University of Mashhad, Theory of Languages and Machines, Project 2 - Model Checker, Phase 3 specification, especially the operational-semantics rules and Option 2 (Loop Invariant), 2026.
- [2] Glynn Winskel, The Formal Semantics of Programming Languages: An Introduction, MIT Press, 1993.
- [3] The Lean Community, Mathematics in Lean, chapters on propositions, inductive types, and induction.
- [4] Benjamin C. Pierce et al., Software Foundations, Logical Foundations, chapters on operational semantics and inductive proofs.